

第13课存储系统 part2

北京交通大学软件学院

严禁复制





04 复制 海量数据存储 (容量扩展)

应用场景

- 定义： 系统需要存储TB、PB甚至EB级别的数据。
- 典型场景：
 - 用户行为日志、传感器数据 (IoT)
 - 历史交易记录、金融市场数据
 - 基因测序数据、科学计算结果
 - 大规模监控视频

严禁复制

使用案例–Facebook

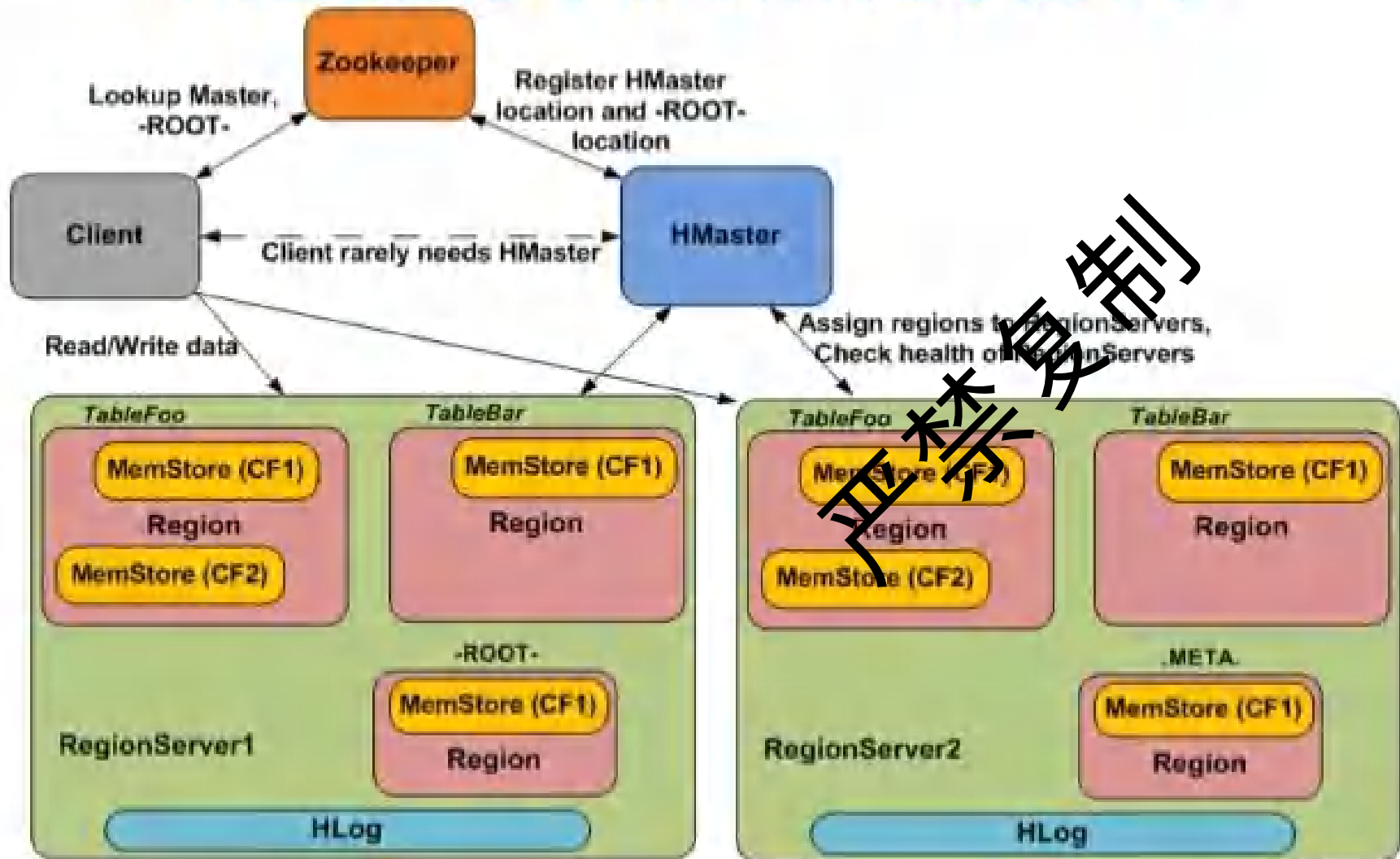
- 数据量：
 - 用户量350M；
 - 消息数120G/月；
- 数据特点：
 - 部分短期数据读写频繁；
 - 不断增长的大部分数据基本保持不变；
- 要求：
 - 一致性
 - 快速处理
 - 高并发
 - 大量数据
- 备选：
 - mysql, Cassandra, hbase



能力细分与相关数据库/技术

- 深化问题：
 - 预期数据增长：数据量预计会增长到什么级别 (TB, PB, EB)?
 - 扩展方式：偏好通过增加节点 (水平扩展) 还是升级硬件 (垂直扩展) 来应对数据增长?
 - 存储成本：单位存储成本是否是重要考量因素？是否有冷热数据分层存储的需求？
- 可能的选项：
 - 水平扩展 (Scale-out):
 - 分布式文件系统 (HDFS)
 - 对象存储 (S3, GCS, MinIO 几乎无限扩展)
 - 支持自动分片的 NoSQL (Cassandra, MongoDB, HBase, DynamoDB)
 - 分布式 SQL (TiDB, CockroachDB)
 - 垂直扩展 (Scale-up, 有物理上限):
 - 传统 RDBMS (可通过升级 CPU, RAM, Disk)
 - 分层存储：
 - 许多数据湖和数据仓库解决方案支持将冷数据归档到更廉价的存储层。

HBase Architecture



关键组件说明:

1. 客户端将请求发送到 Zookeeper, 从中获取负责处理请求的 RegionServer 地址;
2. HBase Master 负责管理 HBase 集群的元数据、区域的负载均衡;
3. 每个 RegionServer 管理若干个 Region, 负责读写操作;
4. Region 是按行键范围划分的, 每个 Region 负责某个连续范围的行键数据;
5. 每个 Region 的数据都会分成多个 HFile 文件, HFile 采用列式存储, 并进行压缩以节省空间;
6. WAL 是 HBase 用来确保数据一致性和容错性的日志。

HBase表结构和普通DB的对比

Customer ID	Name	Address	Product ID	Product Name
1	Paul Walker	US	231	Gallardo
2	Vin Diesel	Brazil	520	Mustang

Table
Row key
Column family
Column qualifiers
Cell
Timestamp:

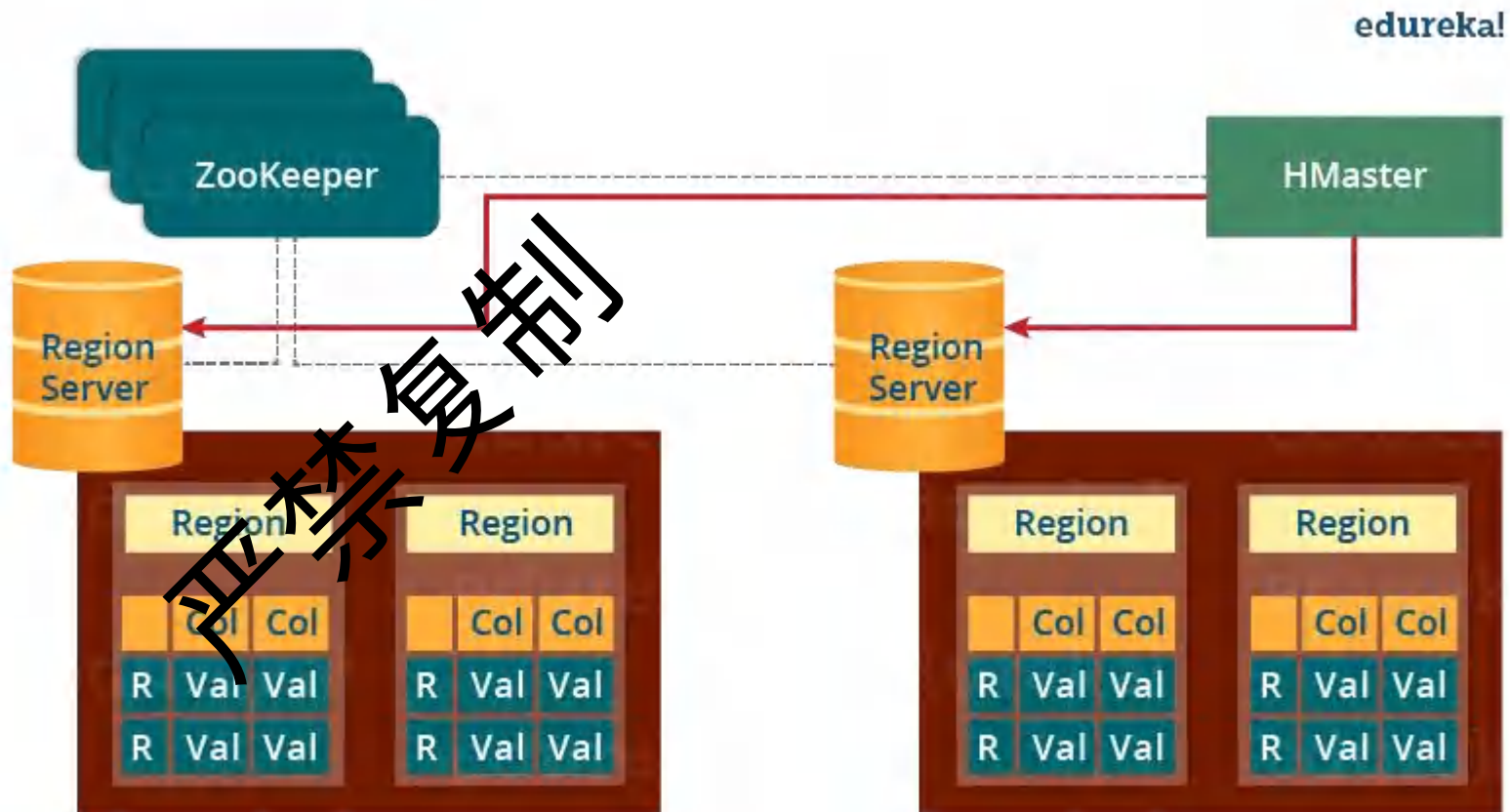
Row Key		Column Family			Column Qualifiers
Row Key		Customers		Products	
Customer ID	Customer Name	City & Country	Product Name	Price	Cell
1	Sam Smith	California, US	Mike	\$500	
2	Arijit Singh	Goa, India	Speakers	\$1000	
3	Ellie Goulding	London, UK	Headphones	\$800	
4	Wiz Khalifa	North Dakota, US	Guitar	\$2500	

<https://www.edureka.co/blog/hbase-architecture/#hbase-architecture-meta-table>

Figure: HBase Table

Hbase关键组件

- HMaster Server,
- Region Server,
- Regions: 属于server
- Zookeeper.



Zookeeper监控

- 每个Region Server以及HMaster服务器都会定期向ZooKeeper发送持续心跳。ZooKeeper会据此检查哪些服务器处于活跃和可用状态。服务器故障通知。

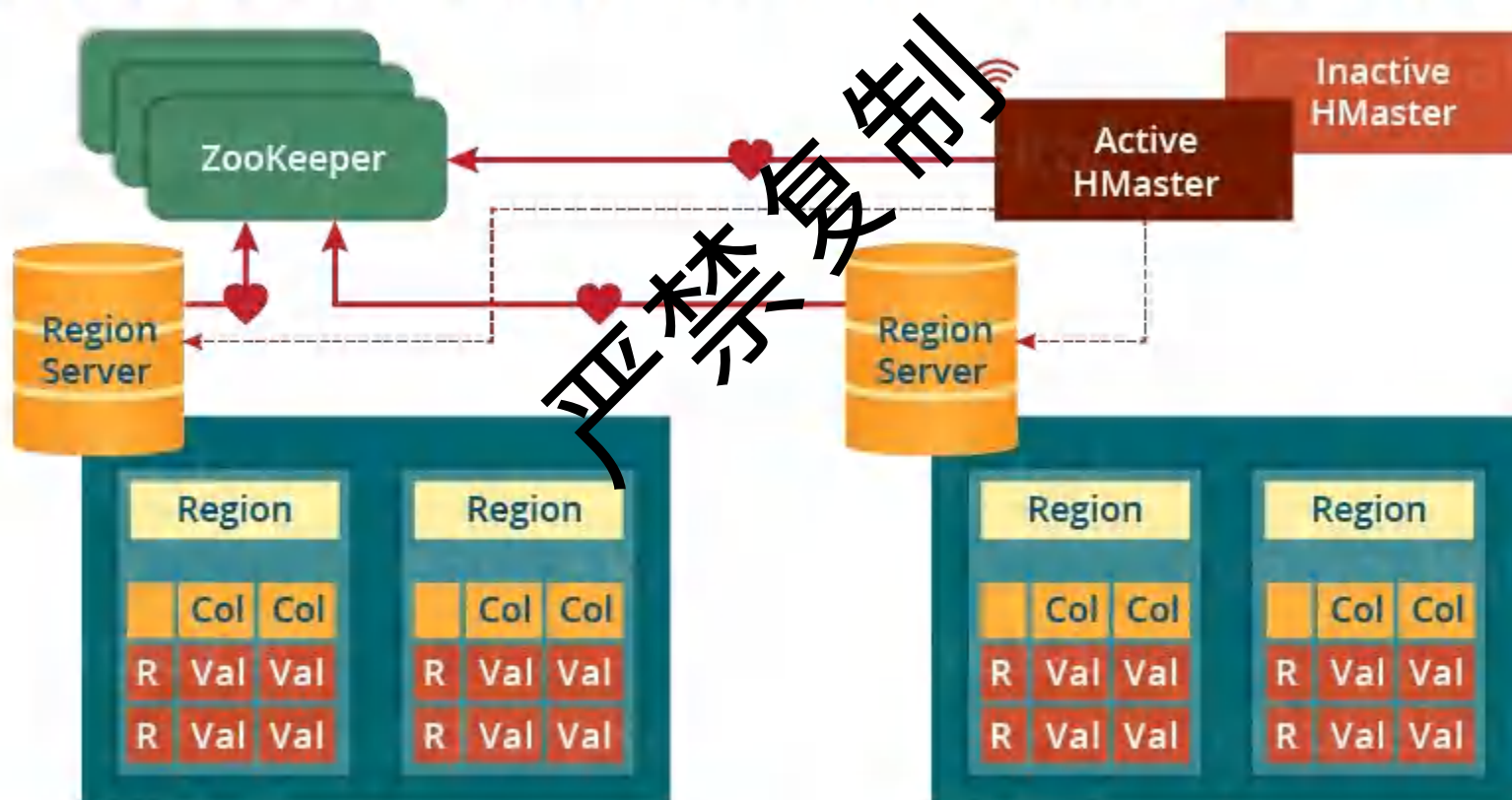


Figure: ZooKeeper as Coordination Service

Zookeeper的查找 (meta)

edureka!

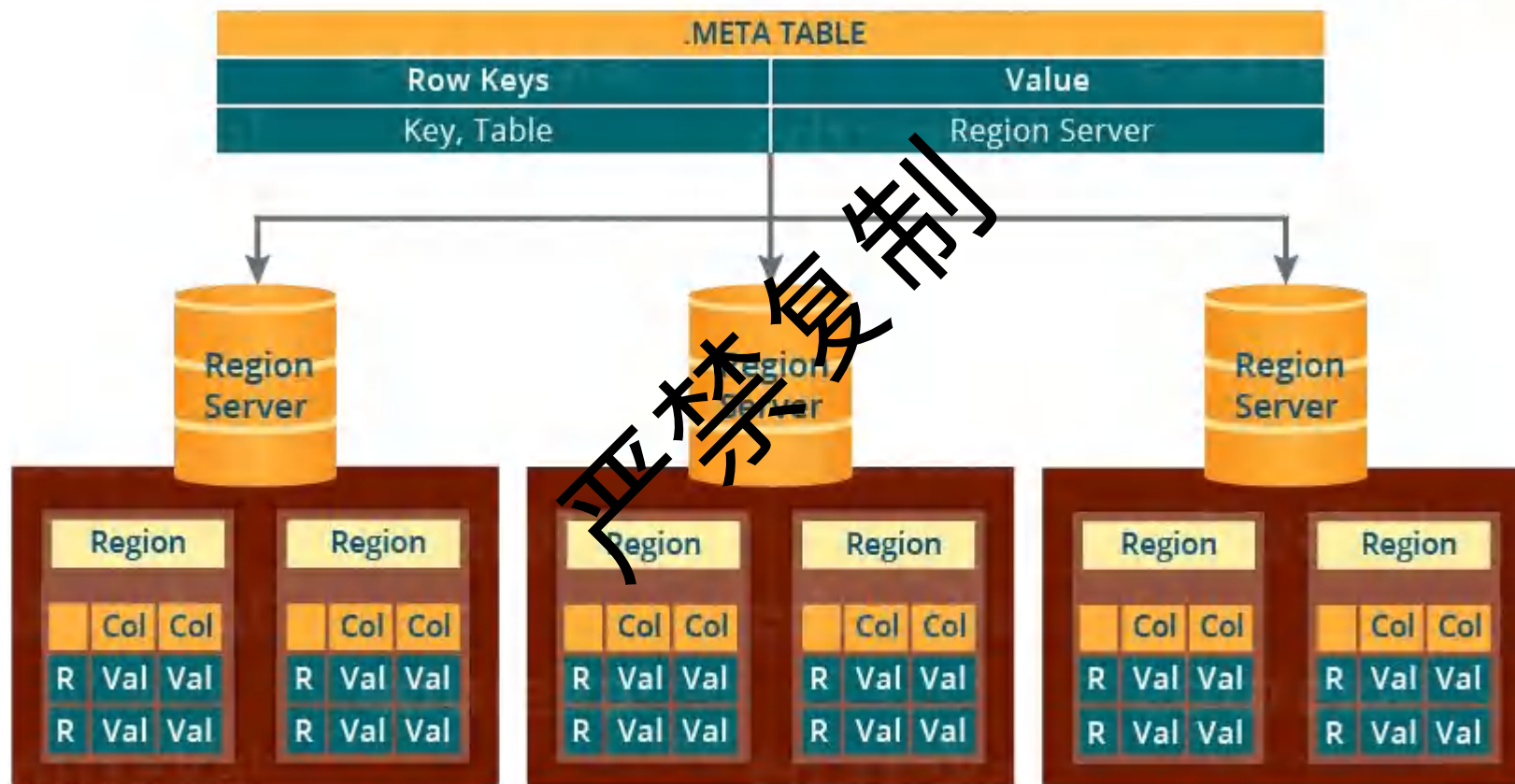


Figure: META Table

Hbase 的写入过程

- 通过WAL来保证一致性，memstore提升写入性能，ACK在memstore写入后返回，memstore的文件达到阈值后写入HFile

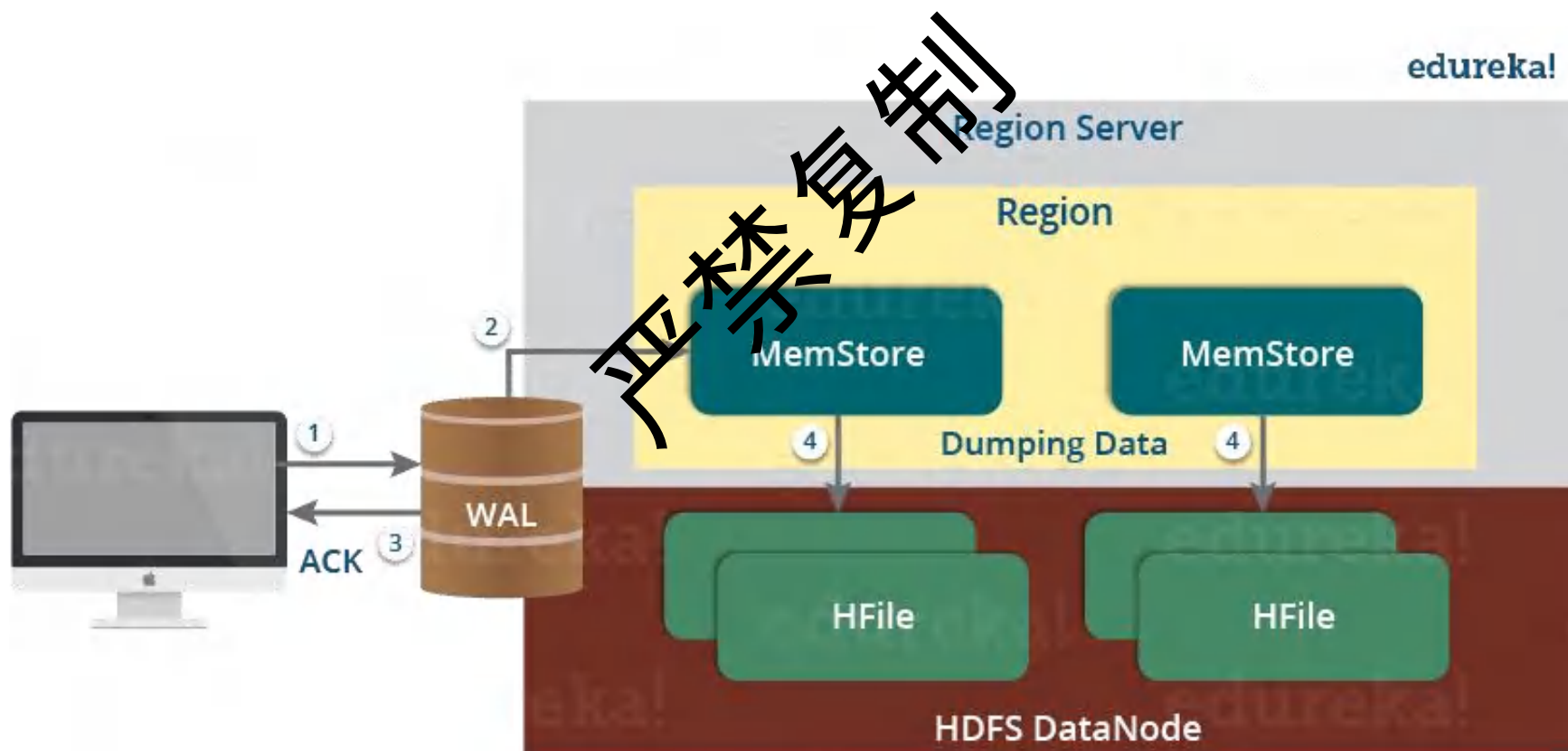
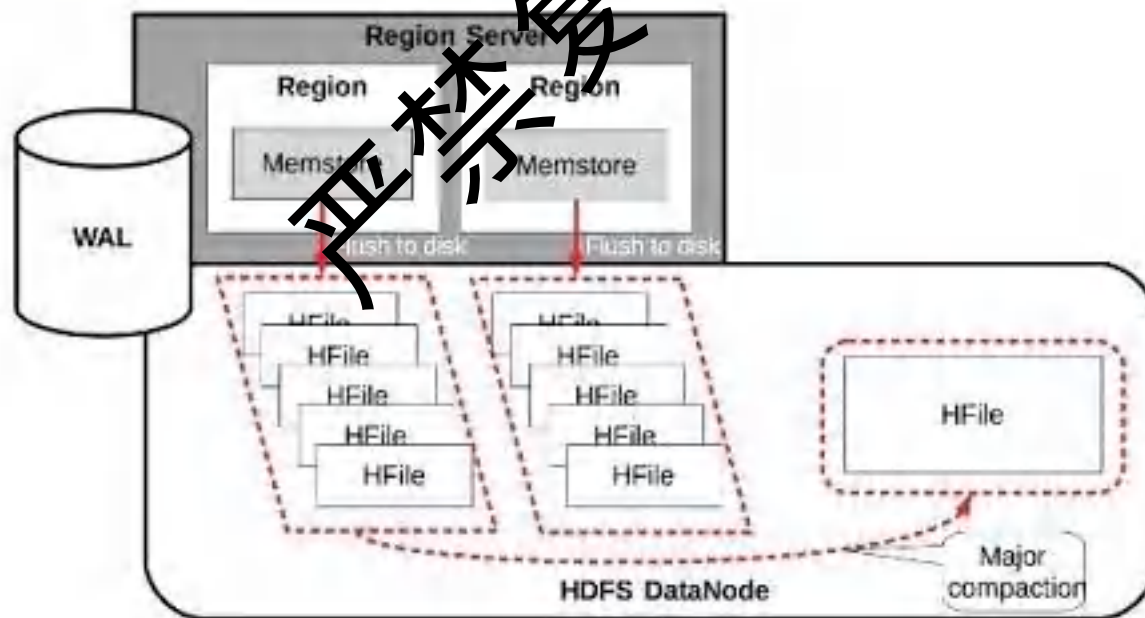


Figure: Write Mechanism in HBase

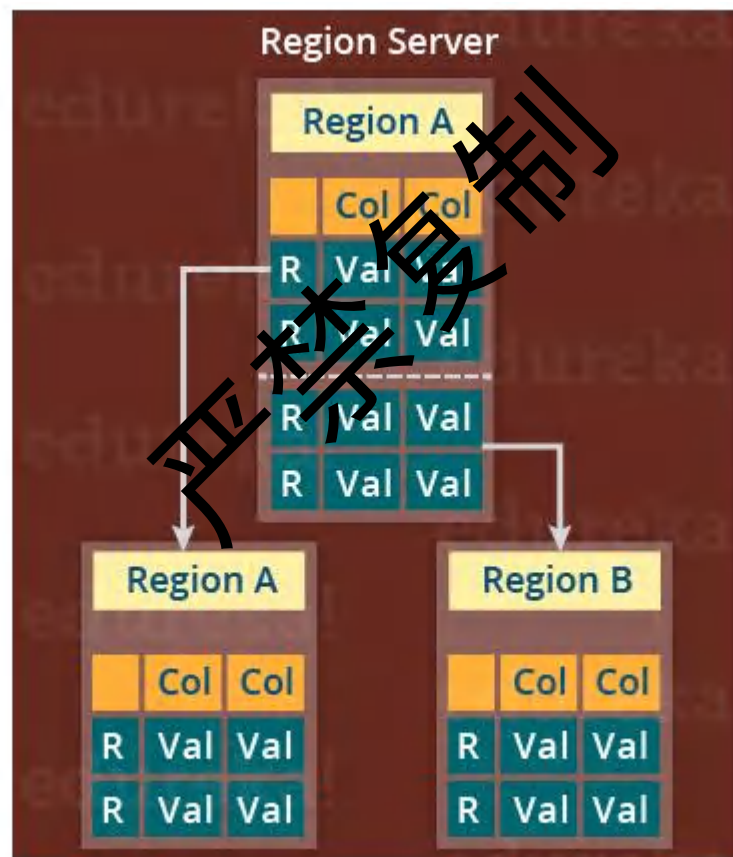
Hbase RegionServer的自动合并

- 小合并：多个小的 HFile 通过合并排序的方式，合并成更大的 HFile。
- 大合并：合并、清除掉所有已删除或过期的数据，从而显著提升读取性能。
- 写入放大：会导致IO和网络的阻塞。



Region Server的自动拆分

- 当region的容量达到阈值之后，自动进行分拆。



edureka!

Figure: Region Split in HBase

HBase的Region Server的失效和恢复

RS失效后：

- 新的RS接手Region；
- 新的RS从WAL中恢复；

问题：

- Region会丢失吗？
- WAL会丢失吗？

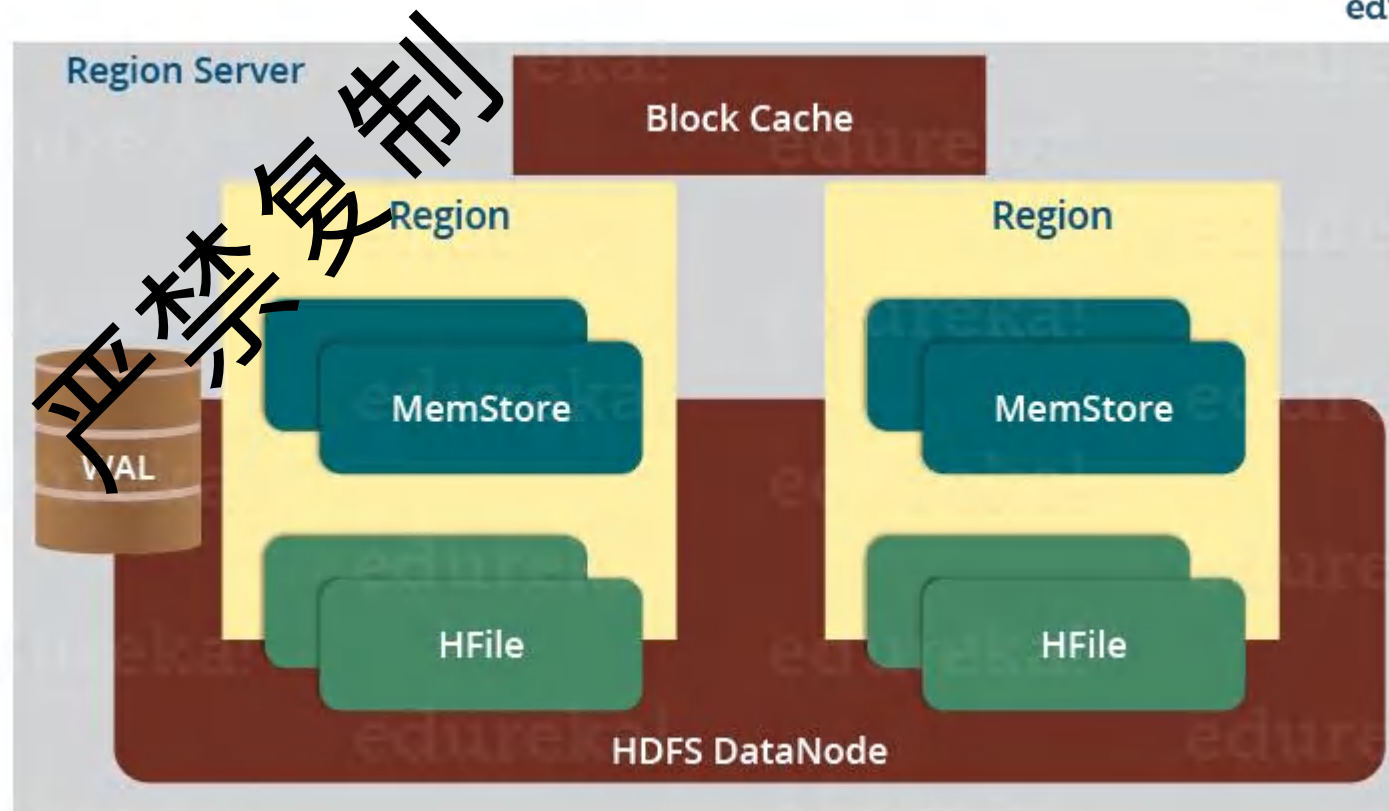
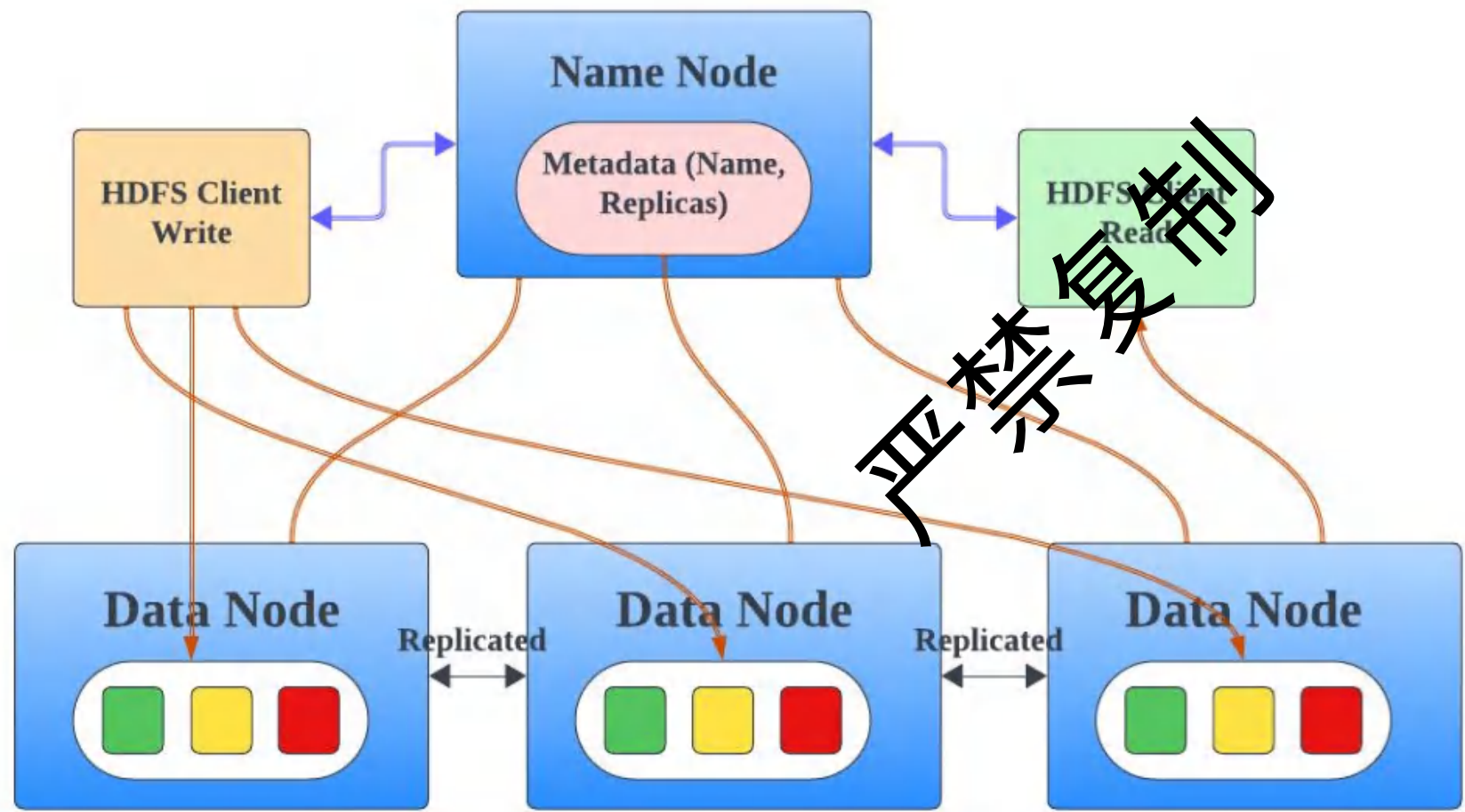


Figure: Region Server Components

HDFS为HBase提供底层支持



特性	描述
文件切块	默认 128MB 每块，按需切分
数据副本	默认 3 个副本，保障容错
写入策略	客户端与多个 DataNode 流水线式写入
读取策略	客户端直接从就近 DataNode 读取
元数据管理	NameNode 记录文件结构、块位置等信息
容错机制	自动复制、坏块检测、NameNode HA

HBase和Mysql的对比

特性	HBase	MySQL
数据量	非常大 (PB 级别)	小型到中型 (GB 到 TB 级别)
数据结构	半结构化、非结构化、模式灵活	结构化、模式定义 (不灵活)
写入量	单节点可达每秒500,000万行, 可水平扩展	每秒5000-50,000行
查询复杂	简单查询、范围扫描、列子集查询	复杂查询、连接、聚合
事务	有限的 ACID 支持	完整的 ACID 支持
可扩展性	高 (水平)	有限 (需要分片才能实现极高的规模)
实时访问	查询延迟毫秒~秒级	查询延迟毫秒级
用例	数据仓库、日志分析、时间序列数据、物联网数据	OLTP 应用程序、电子商务、内容管理、金融系统



05 复制 全球海量数据分析

应用场景

- 定义： 需要对大规模数据集进行复杂的查询、聚合、统计和挖掘。
- 典型场景：
 - 商业智能 (BI) 报表、仪表盘
 - 用户画像分析、精准营销
 - 欺诈检测、风险控制模型训练
 - 机器学习/深度学习数据预处理和模型训练

严禁复制

实时分析的类型

- 交互式 Ad-hoc 查询
 - 强调灵活、即时的分析体验，适合临时性、探索性查询。
 - 秒级~分钟级
 - 数据探索、报表、临时分析
- 批处理 ETL (Extract-Transform-Load)
 - 强调高吞吐、复杂处理，适合定期离线汇总与加工。
 - 分钟~小时级
 - 离线汇总、数据仓库
- 实时流分析
 - 强调低延迟、实时性，适合监控、预警、实时决策
 - 毫秒~秒级
 - 监控报警、实时推荐、流仪表盘

严禁复制

实时分析的类型

- 交互式 Ad-hoc 查询
 - 常用工具如 Presto、Impala、ClickHouse、Druid、阿里云 AnalyticDB 等
- 批处理 ETL (Extract-Transform-Load)
 - 常用工具如 Hadoop MapReduce、Spark、Flink (批模式)、阿里云 DataWorks 等。
- 实时流分析
 - 常用工具如 Apache Flink、Spark Streaming、Storm、阿里云实时计算 Flink 版等

禁止转载

能力细分与相关数据库/技术

特性	Presto	Impala	ClickHouse	Druid
存储方式	外部多源	外部 (Hive/HDFS)	内部列存	内部列存 +多级索引
联表能力 JOIN	强（多源联邦）	强	一般（支持但性能一般）	弱（不适合复杂JOIN）
实时性	取决于底层存储	取决于底层存储	秒级 (批量导入为主)	毫秒级 (原生流式摄入)
高并发	一般	一般	很强	很强
典型场景	跨源分析、 临时查询	大数据仓库分析	实时报表、 行为分析	实时仪表盘、 监控、 时序分析

Trino-分布式查询引擎

Presto 和 Trino 最初由 Facebook 的 Martin、Dain、David 和 Eric Hwang 设计和开发，旨在能够让数据分析师在 Apache Hadoop 中的大型数据仓库上执行交互式查询。

2020 年 12 月，PrestoSQL 更名为 Trino。作为品牌重塑的一部分，Trino 软件基金会、代码库和所有其他 PrestoSQL 资产都已经按新品牌重命名



Presto的整体架构

1. 构建逻辑plan

1. 构建逻辑执行计划，并进行优化（如谓词下推、连接顺序调整、剪枝等）。

2. 构建物理plan

1. 将逻辑执行计划转换为可实际执行的物理计划（Stage + Task）。

3. 任务分发

1. 将不同的执行阶段（Stage）分发给合适的 Worker 节点执行。

4. 结果汇总

5. worker资源管理

1. 管控内存、CPU 使用限制，确保作业公平性与高可用性。

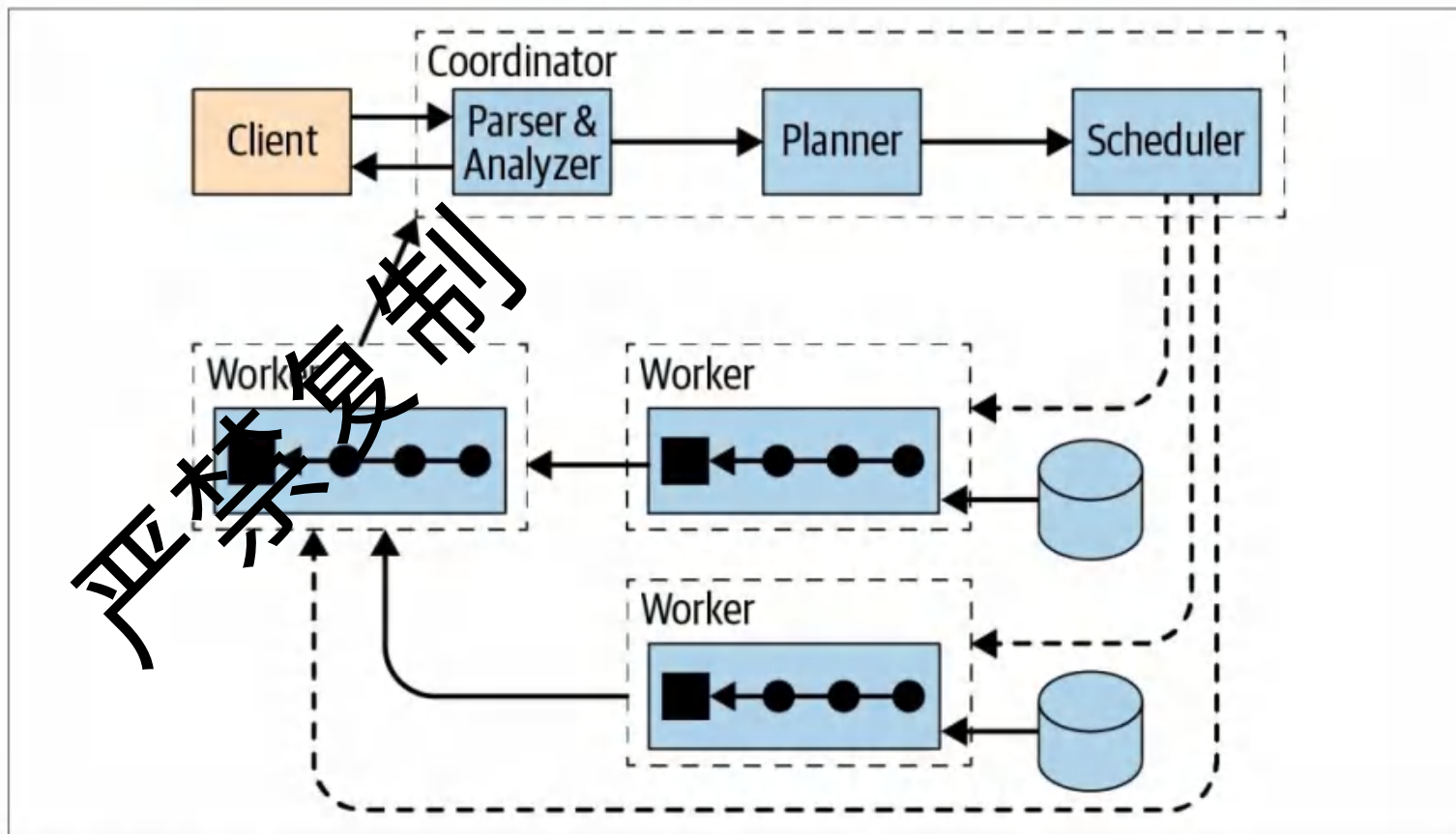


Figure 4-1. Trino architecture overview with coordinator and workers

关键技术实现简述

- MPP (Massively Parallel Processing) 架构：
 - 无共享架构，数据和计算分散到多个节点并行执行。
- 列式存储：
 - 按列存储数据，利于压缩，且分析查询通常只涉及部分列，减少I/O。
- 向量化执行：
 - CPU一次处理一批数据（一个向量）而非单条记录，利用SIMD指令。
- 查询优化器：
 - 基于成本估算生成高效的执行计划。
- 谓词下推 (Predicate Pushdown):
 - 尽可能早地过滤数据。

MPP架构

- MPP的定义
- MPP的使用场景
- MPP的优化算法概述

严禁复制

MPP架构的定义

- MPP (Massively Parallel Processing) 架构
 - 大规模并行处理架构，数据库管理系统架构，核心思想“Shared-Nothing”（无共享）
 - 每个节点独立的CPU、内存和存储
 - 节点之间通过高速网络互联，不共享任何资源
- 用途：主要用于数据仓库和分析领域
 - 能够处理海量数据
 - 提供极高的查询性能
- 对比：
 - SMP：单机多核，容量有限
 - Map reduce：容量无限，但是延迟较高

严禁复制

MPP的优势和劣势

- 优点：

- 线性扩展性： 增加节点即可扩展存储和计算能力，数据量PB EB级别。
- 高可用性： 单个节点的故障不会导致整个系统停机。
- 高吞吐量： 非常适合复杂的分析型查询（OLAP），可以并行处理大规模数据。
- 成本效益： 可以使用廉价的商品硬件构建。

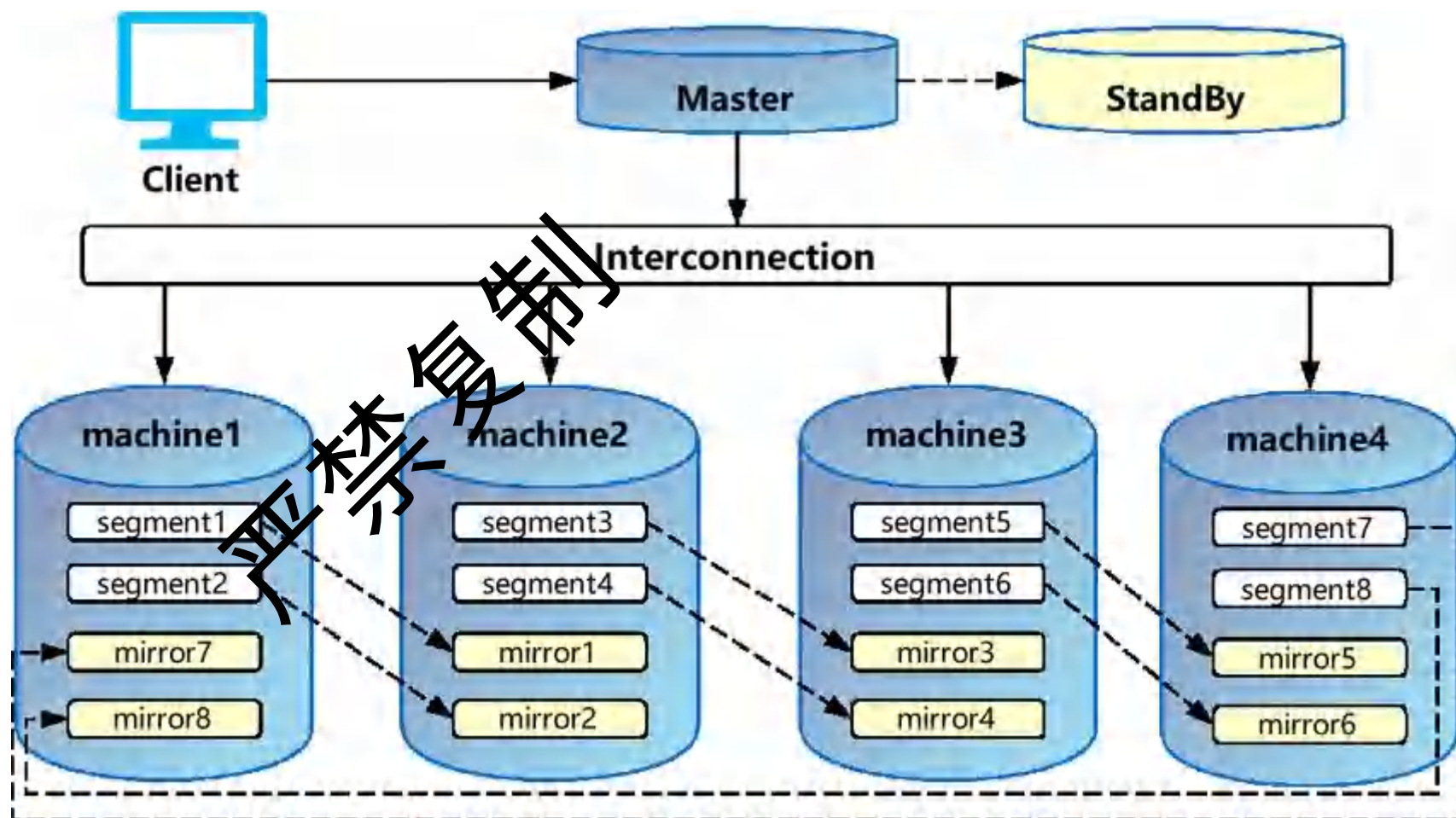
- 缺点：

- 数据分布复杂：
 - 数据需要合理地分布到各个节点上，
 - 查询时的负载均衡和避免数据倾斜。
- 查询优化复杂：
 - 查询优化器需要能够理解数据分布，并生成有效的并行执行计划。
- 节点间通信开销：
 - 数据在节点间传输时会产生网络延迟。
- 管理复杂：
 - 需要管理大量的节点和网络。

严禁复制

MPP架构

- 协调器节点
 - 逻辑执行计划
 - 物理执行计划
 - 任务执行
 - 集群管理
- 段节点
 - 数据分片存储
 - 扫描、过滤、排序、join
 - IO吞吐量50G/s (10G/s)
- 高速互联网络
 - 100–200Gpbs
- 存储子系统
 - 持久化数据
 - 高并发I/O



MPP的场景例子

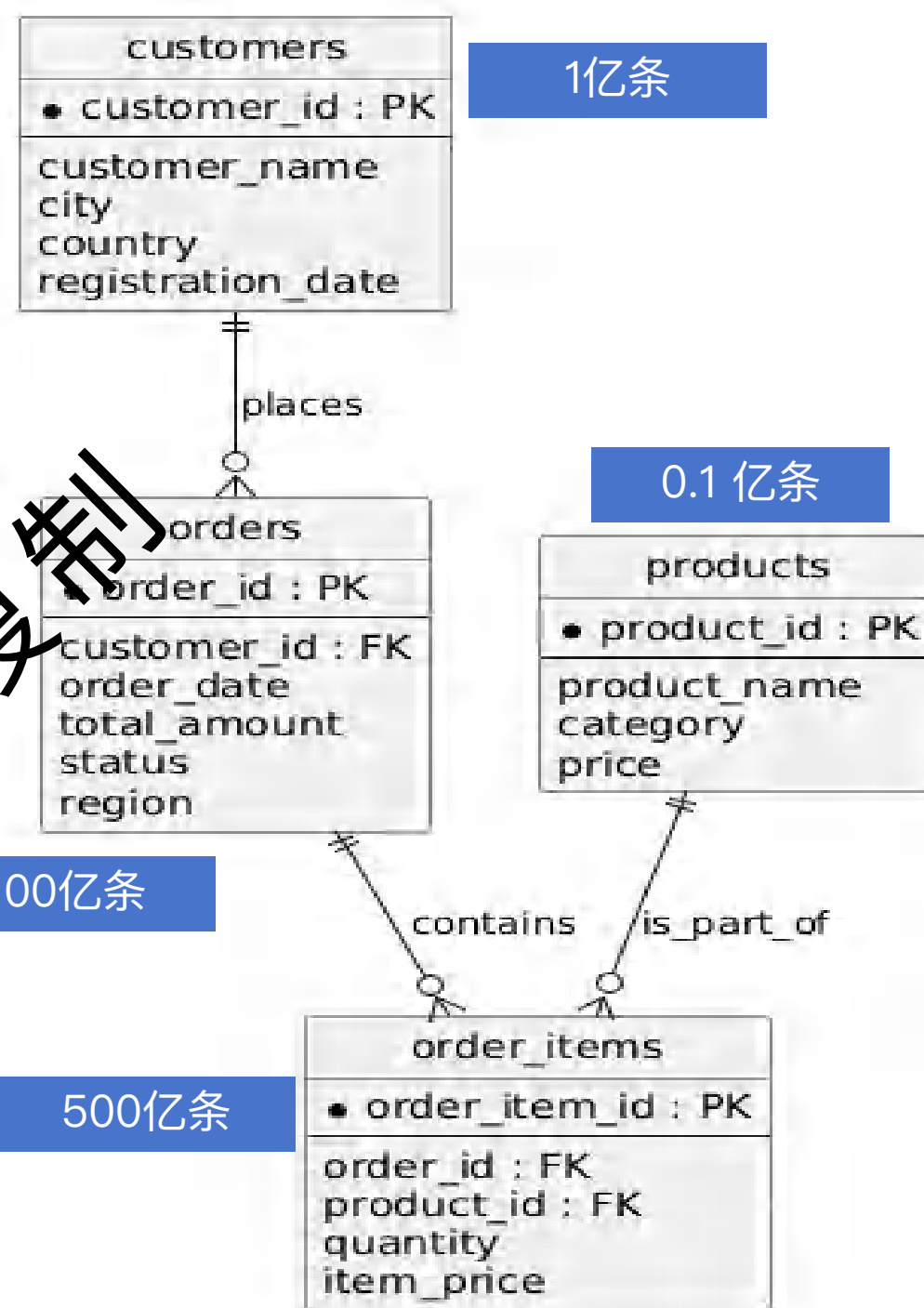
- 背景:

- 假设一个大型的电商交易数据集。
- 数据存储在3台机器上

- 查询:

- 简单数据过滤与局部聚合
- 多表关联与分布式聚合
- 复杂多表关联、子查询与窗口函数

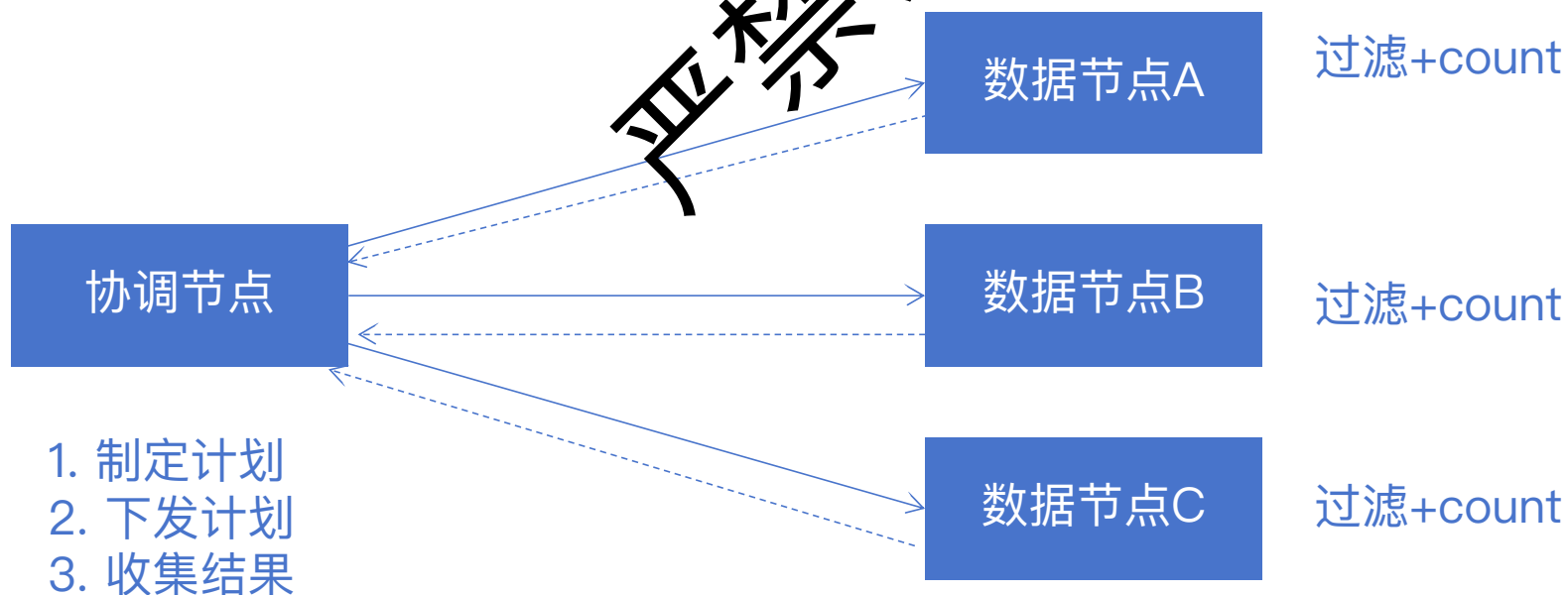
严禁复制



简单场景的处理过程

- 查询目标： 找出2023年Q3所有完成订单中，总金额超过500美元的订单数量
- 处理过程：

```
SELECT
    COUNT(order_id) AS high_value_orders_count
FROM
    orders
WHERE
    order_date >= '2023-07-01' AND order_date <= '2023-09-30'
    AND status = 'completed'
    AND total_amount > 500;
```

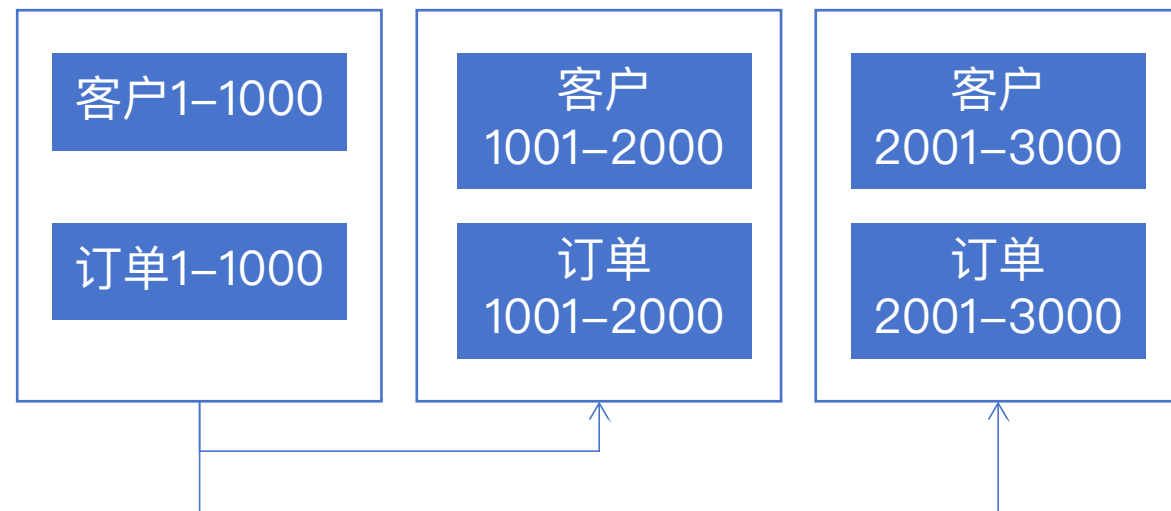


场景2 多表关联与分布式聚合

- 查询目标：
 - 统计每个国家在2023年的客户总销售额
 - 注册日期早于2020年
 - 并按总销售额降序排列

```
SELECT
    c.country,
    SUM(o.total_amount) AS total_sales
FROM
    orders AS o
JOIN
    customers AS c ON o.customer_id = c.customer_id
WHERE
    o.order_date >= '2023-01-01' AND o.order_date <= '2023-12-31'
    AND c.registration_date < '2020-01-01'
GROUP BY
    c.country
ORDER BY
    total_sales DESC;
```

1. 协调节点下发任务
2. 本地扫描
3. orders和customers进行JOIN,
 1. 都以customer_id作为分布键
 2. 否则需要进行节点数据传递
4. JOIN之后按照Country字段汇总
5. 将聚合结果发回到协调节点



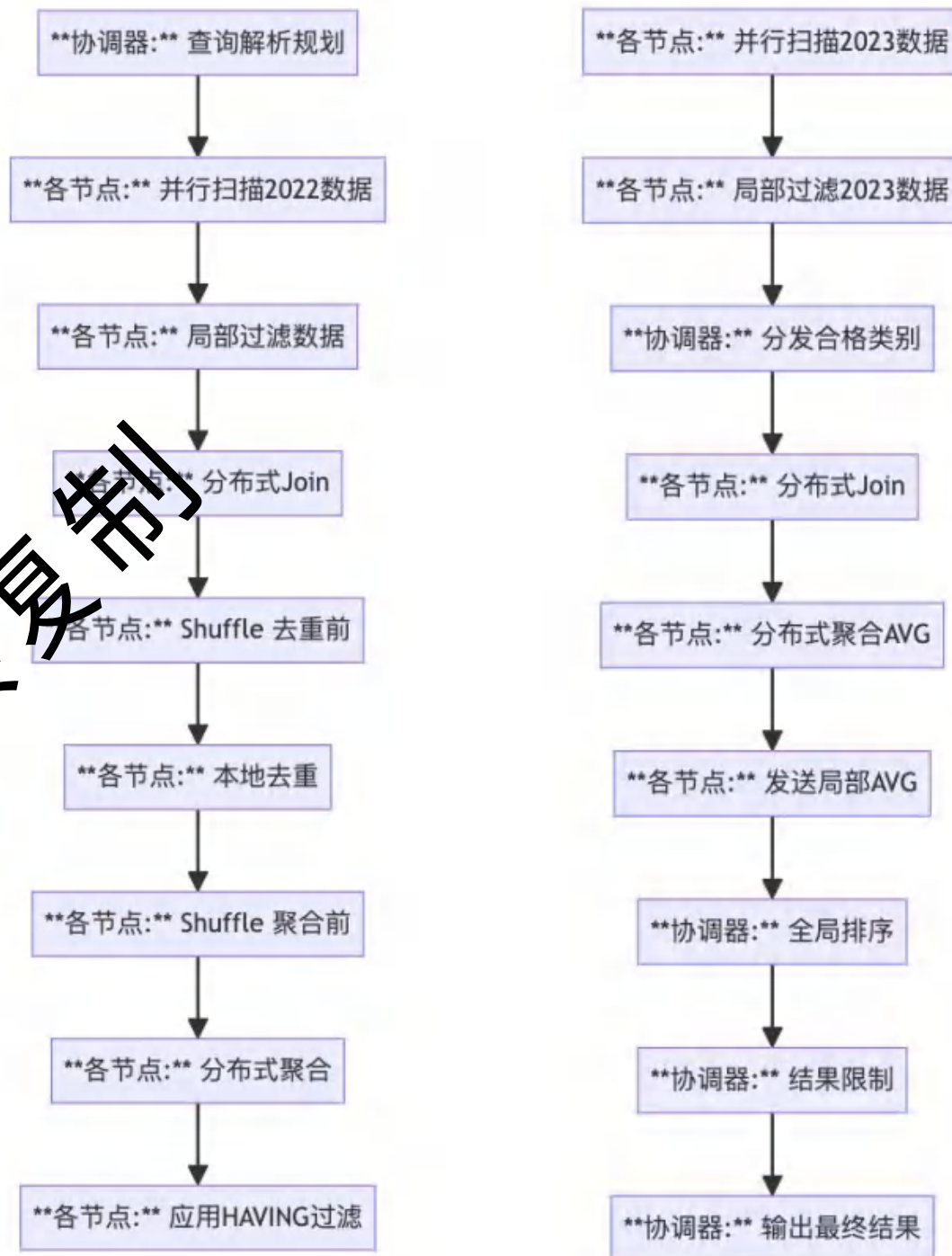
场景3 复杂多表关联、子查询与窗口函数

- 目标:

- 找出2023年**平均订单金额**最高的3个产品类别,
- 但仅限那些在过去一年
- 1000个不同客户购买过

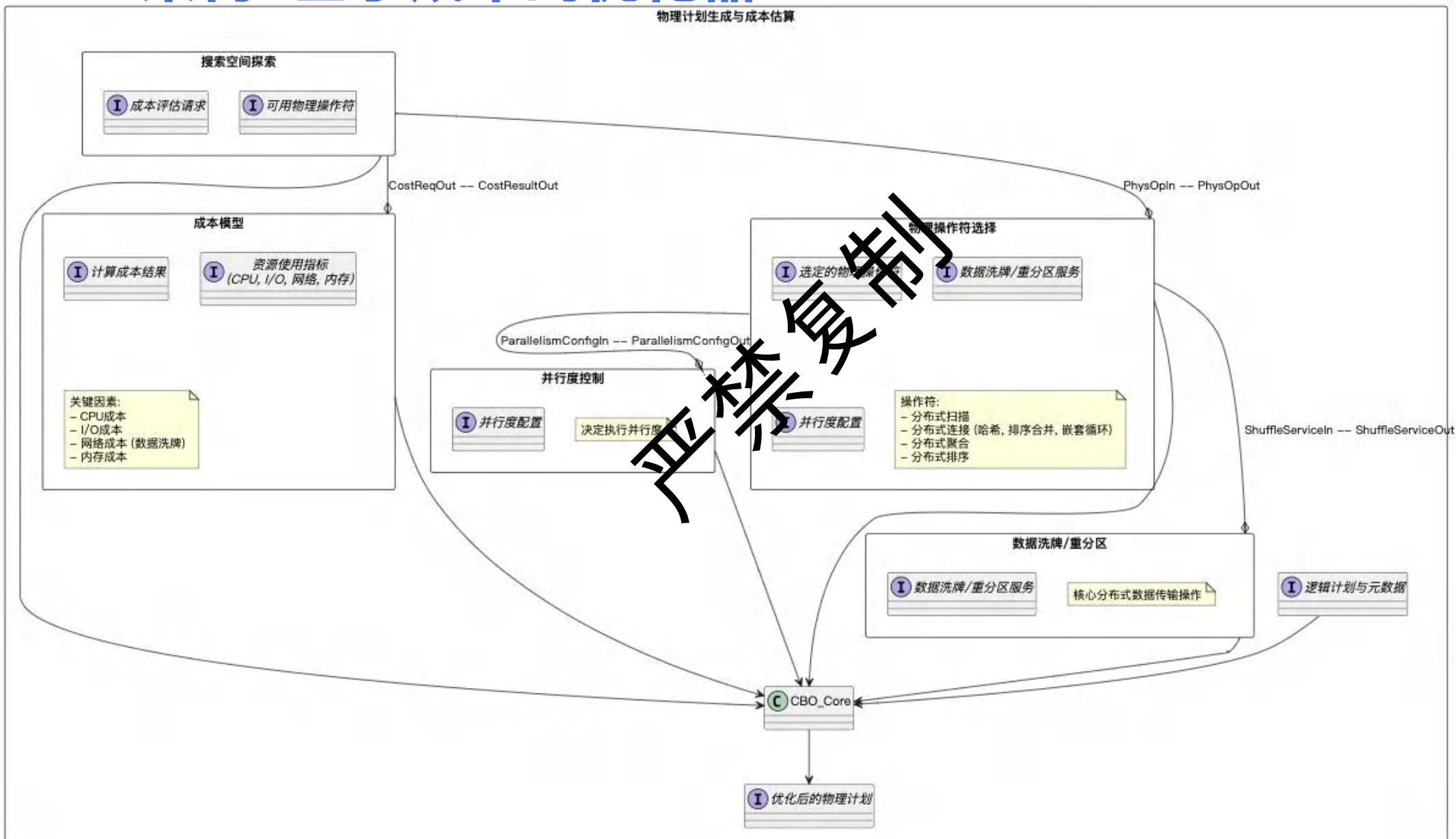
- 瓶颈:

- 多阶段的数据Shuffle (涉及大量的网络传输和磁盘I/O) 、
- 中间结果的临时存储和读取、
- 以及CPU进行复杂的哈希、Join和聚合计算。



MPP架构-基于成本的优化器CBO

物理计划生成与成本估算



MPP架构CBO算法的原则

- 最小化数据移动 (Minimize Data Movement / Shuffle):
 - 这是MPP查询优化的首要目标。优化器会优先选择不需要或只需要少量数据传输的计划（例如，利用数据已经共置Collocated的特性，或选择广播小表）。
- 最大化并行度 (Maximize Parallelism):
 - 确保查询的各个部分都能在尽可能多的节点上并行执行。
- 充分利用数据分布 (Leverage Data Distribution):
 - 考虑表在集群中的分区方式（哈希、范围、复制等），选择能够避免或减少数据重分区的操作。
- 准确的统计信息 (Accurate Statistics):
 - 估算成本的基础，直接影响优化器选择计划的质量。
- 推下操作 (Pushdown Operations):
 - 尽可能早地执行过滤、投影等操作，减少处理的数据量。



06 复制
任务增加字段

应用场景

- 定义： 系统需要能够轻松适应数据结构的变化，如添加新字段、修改字段类型，而不需要大规模停机或复杂的数据迁移。
- 典型场景：
 - 快速迭代的互联网产品 (A/B 测试, 新功能频繁上线)
 - 用户自定义字段 (CRM, SaaS平台)
 - 多租户系统中，不同租户可能有不同的数据扩展需求
 - 数据来源多样，结构不统一

禁止转载

进一步深化的问题

- Schema变更的频率有多高?
- 变更时是否能容忍服务中断或性能下降?
- 变更对现有查询和应用代码的影响如何评估?
- 如何管理和版本化不同的Schema?

禁止复制

能力细分与相关数据库/技术

- Schema-less

- 意味着数据存储本身不强制预先定义数据的结构（模式）
- 优点：当新的数据字段出现时，无需修改现有模式
- 缺点：由于没有强制模式，数据质量难以保证，类型不一致或数据缺失
- 工具：一些 NoSQL 数据库，如 MongoDB、Couchbase 等

- Schema-on-read

- 意味着数据的结构（模式）在读取数据时被定义或应用。数据存储本身可能没有强制的模式，或者模式非常灵活。
- 优点：允许在数据摄取后，根据实际的分析需求来定义模式
- 缺点：影响查询性能，需要特定的工具
- 工具：Apache Hive、Apache Spark SQL 等

MongoDB如何实现无模式读写

```
// 2. 获取数据库和集合
MongoDatabase database = mongoClient.getDatabase("mydatabase");
MongoCollection<Document> collection = database.getCollection("users");

// 3. 创建 Java 对象
Person person = new Person("Alice", 30, "alice@example.com");

// 4. 将 Java 对象转换为 MongoDB Document
Document document = new Document()
    .append("name", person.getName())
    .append("age", person.getAge())
    .append("email", person.getEmail());

// 5. 插入文档
collection.insertOne(document);
```

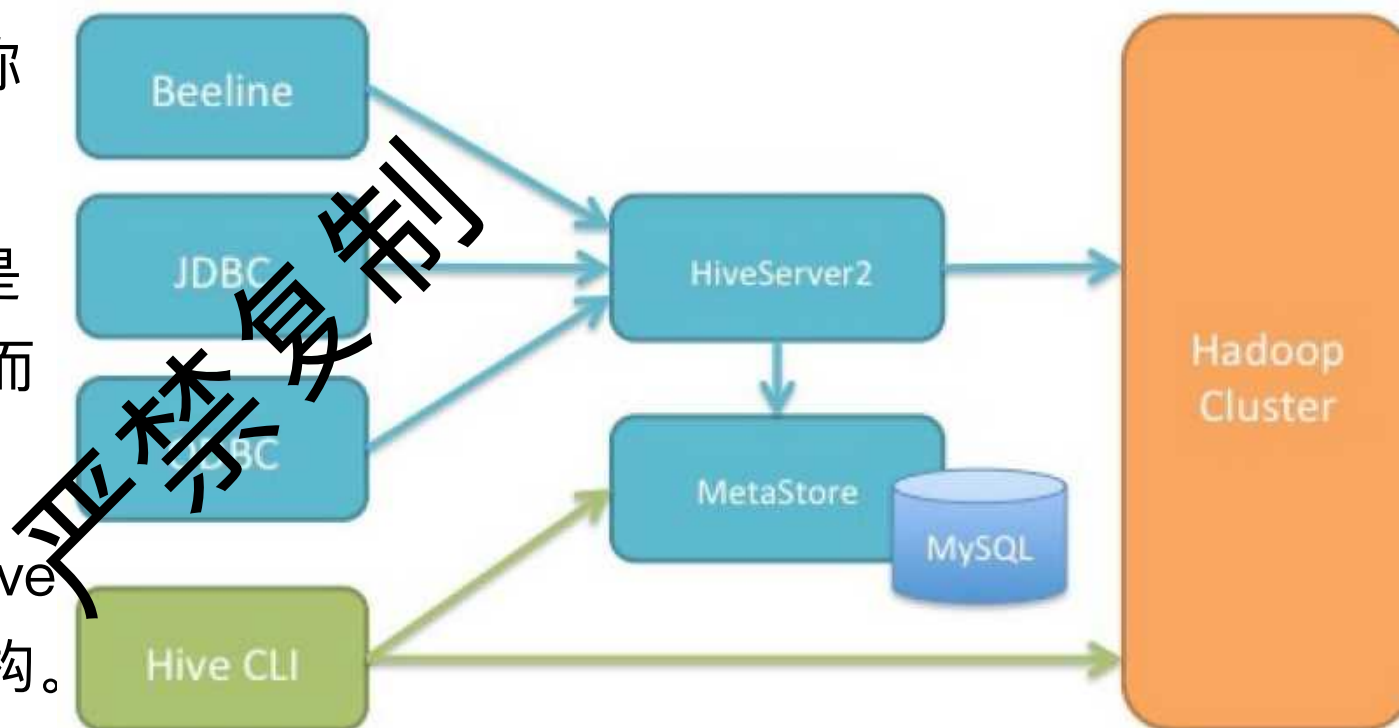
```
// 2. 获取数据库和集合
MongoDatabase database = mongoClient.getDatabase("mydatabase");
MongoCollection<Document> collection = database.getCollection("users");

// 3. 遍历文档
try (MongoCursor<Document> cursor = collection.find().iterator()) {
    while (cursor.hasNext()) {
        Document document = cursor.next();

        // 4. 从 Document 中读取数据
        String name = document.getString("name");
        int age = document.getInteger("age");
        String email = document.getString("email");
        String position = document.getString("position");
    }
}
```

apache hive如何实现schema on read

- 通过使用元数据存储（Metastore）来支持 Schema-on-Read，也称为动态模式。
- Schema-on-Read 的核心思想是在数据写入时不对其进行验证，而是在查询时才应用模式。
- 可以将各种格式的数据加载到 Hive 表中，而无需预先定义数据的结构。
- Hive 在查询时根据 Metastore 中定义的模式来解析数据。





7 复制
严格复制
访问量水平扩展

应用场景

- 定义： 当用户请求量（QPS/TPS）激增时，系统能够通过增加更多节点来分摊负载，保持高性能和高可用。
- 典型场景：
 - 电商促销活动（双十一、黑五）
 - 突发热点新闻、病毒式营销事件
 - 高并发API网关后端服务

严禁复制

进一步深化的问题

- 读写负载如何分布？
 - 读多写少 vs. 写多读少 vs. 读写均衡
- 扩展时如何保证数据一致性？
 - 特别是写操作的扩展
- 负载均衡策略如何设计？
- 缓存策略在应对访问量激增时的作用和挑战？
 - 缓存穿透、雪崩、击穿

禁止转载

相关数据库/技术

- 读写分离架构：
 - MySQL/PostgreSQL 主从复制 + 代理 (ProxySQL, MaxScale)
- 分布式缓存集群：
 - Redis Cluster, Memcached (配合客户端分片)
- 原生支持水平扩展的数据库：
 - NoSQL: Cassandra, MongoDB (分片), DynamoDB
 - NewSQL: TiDB, CockroachDB
- 应用层分片/路由

禁止转载

newsq1

- NewSQL 数据库

- 提供关系型数据库的 ACID 事务一致性和 SQL 兼容性；
- 同时具备 NoSQL 数据库的水平扩展性和高可用性。

- NewSQL VS NoSQL

- 数据模型和查询语言
- 事务和一致性保证
- 强一致性、复杂查询和事务处理，同时又要求海量数据存储和高并发读写的场景

- NewSQL VS RDB

- 水平扩展 VS 垂直扩展
- 高可用性，容灾能力

TiDB vs CockroachDB

- 技术栈

- mysql VS postgres

- 业务需求

- TiDB

- 大规模OLTP，实时数据分析

- CockroachDB

- 全球性应用，跨地域的强一致性和低延迟

- 运维

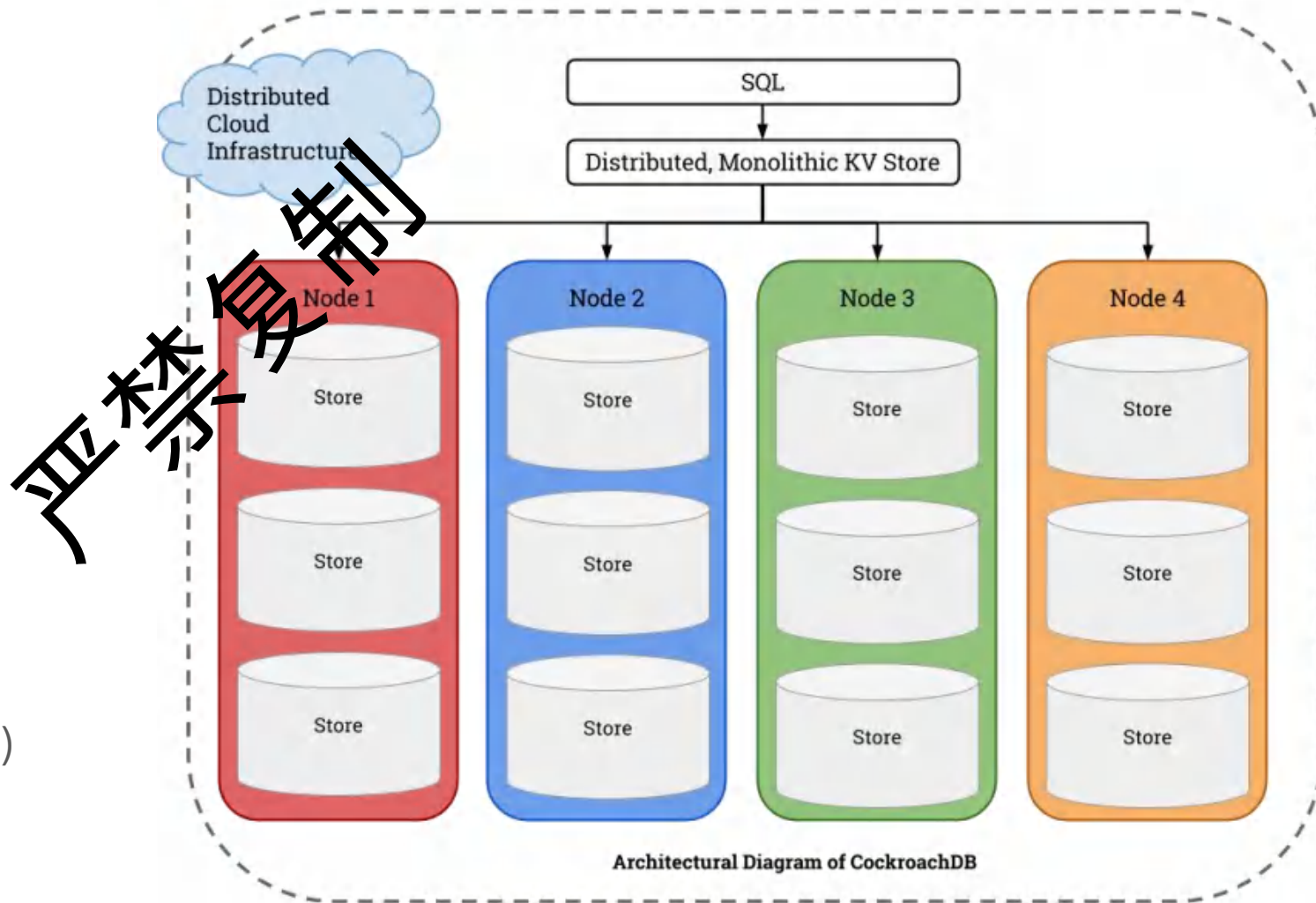
- CockroachDB更简单
- TiDB分层更多

禁止转载

CockroachDB如何实现跨地域事务

<https://www.cockroachlabs.com/blog/the-new-stack-meet-cockroachdb-the-resilient-sql-database/>

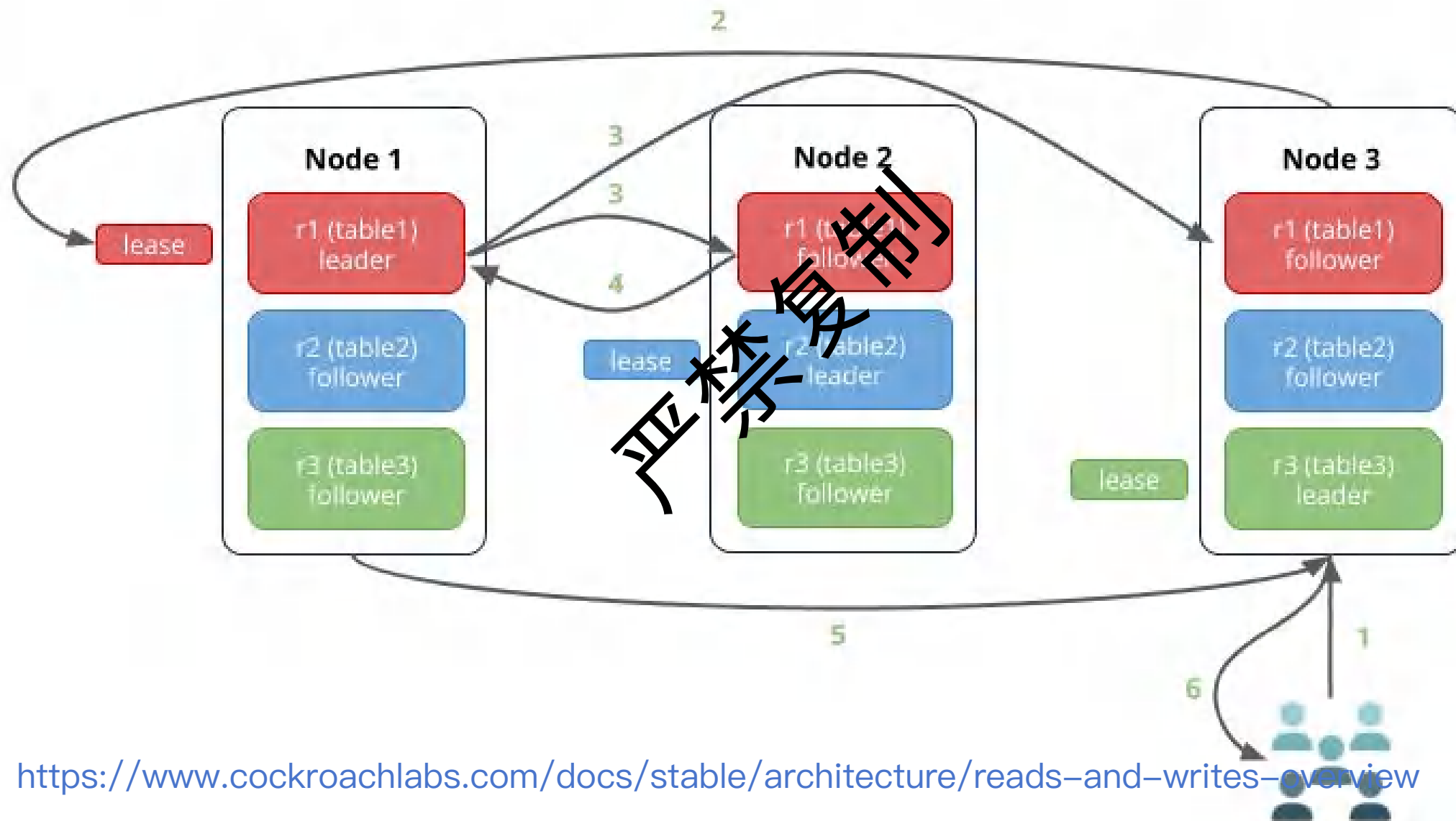
- SQL Layer (SQL 层)
- 分布式事务协调器
 - 启动事务、分配事务 ID、
 - 两阶段提交 (2PC) 协议协调涉及到的 (Range) ,
 - 处理冲突, 以及最终提交或回滚事务
- HLC 混合逻辑时钟
 - 全球范围内是单调递增且几乎唯一的
- 键值存储层
 - 处理并发读写,
 - 切分成小的 (64MB) 分片 (Ranges)
 - 每个 Range 都运行一个 Raft 组
- MVCC 多版本并发控制



CockroachDB如何实现跨地域事务

- 两个事务都尝试执行：
 - `UPDATE products SET stock = stock - 1 WHERE shoe_id = X AND stock > 0;`
 - `INSERT INTO orders (...) VALUES (...);`
- 对库存的更新（核心瓶颈）：
 - 无论 Txn_L 还是 Txn_B，它们的 UPDATE 请求都必须发送到纽约的 Raft Leader。
 - 假设 Txn_L 的请求先到达纽约 Leader。Leader 会在 MVCC 层检查并尝试减少库存。
 - 如果 Txn_L 成功将库存从 1 减少到 0，这个更新会通过 Raft 协议复制到伦敦和北京的副本（多数派确认，即纽约+伦敦或纽约+北京）。
 - 紧接着，Txn_B 的请求到达纽约 Leader。版本冲突（或者库存是0）。
 - MVCC 检测到写冲突，Txn_B 会被告知冲突，通常导致 Txn_B 重试 (Retry)。在重试时，Txn_B 会看到更新后的库存为 0，从而根据应用逻辑判断无法购买，最终失败或退回。
 - 订单创建：同时，两个事务也会尝试创建订单。如果下单成功，对应的 INSERT 也会写入。

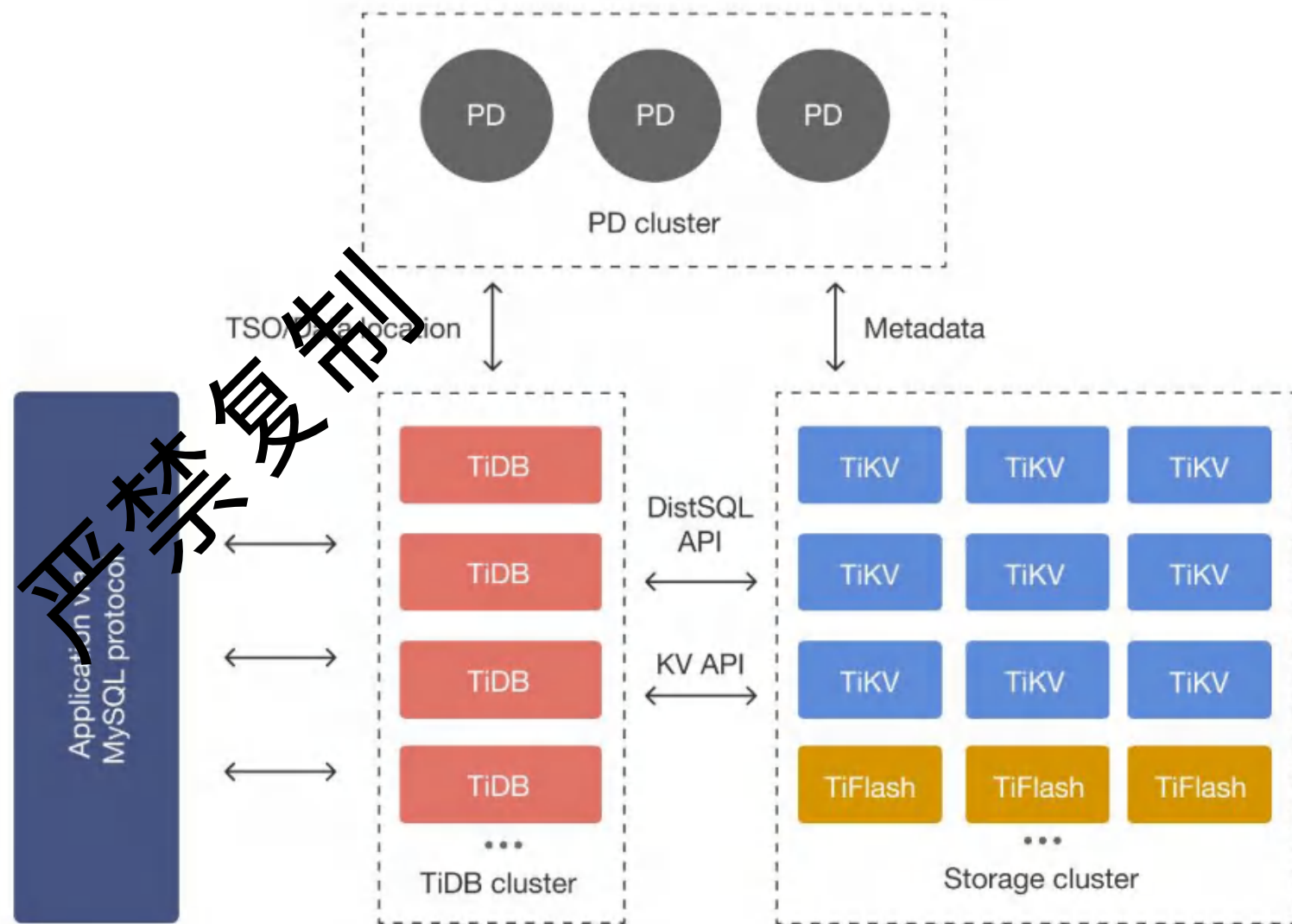
CockroachDB写入：Raft协议保持强一致性



<https://www.cockroachlabs.com/docs/stable/architecture/reads-and-writes-overview>

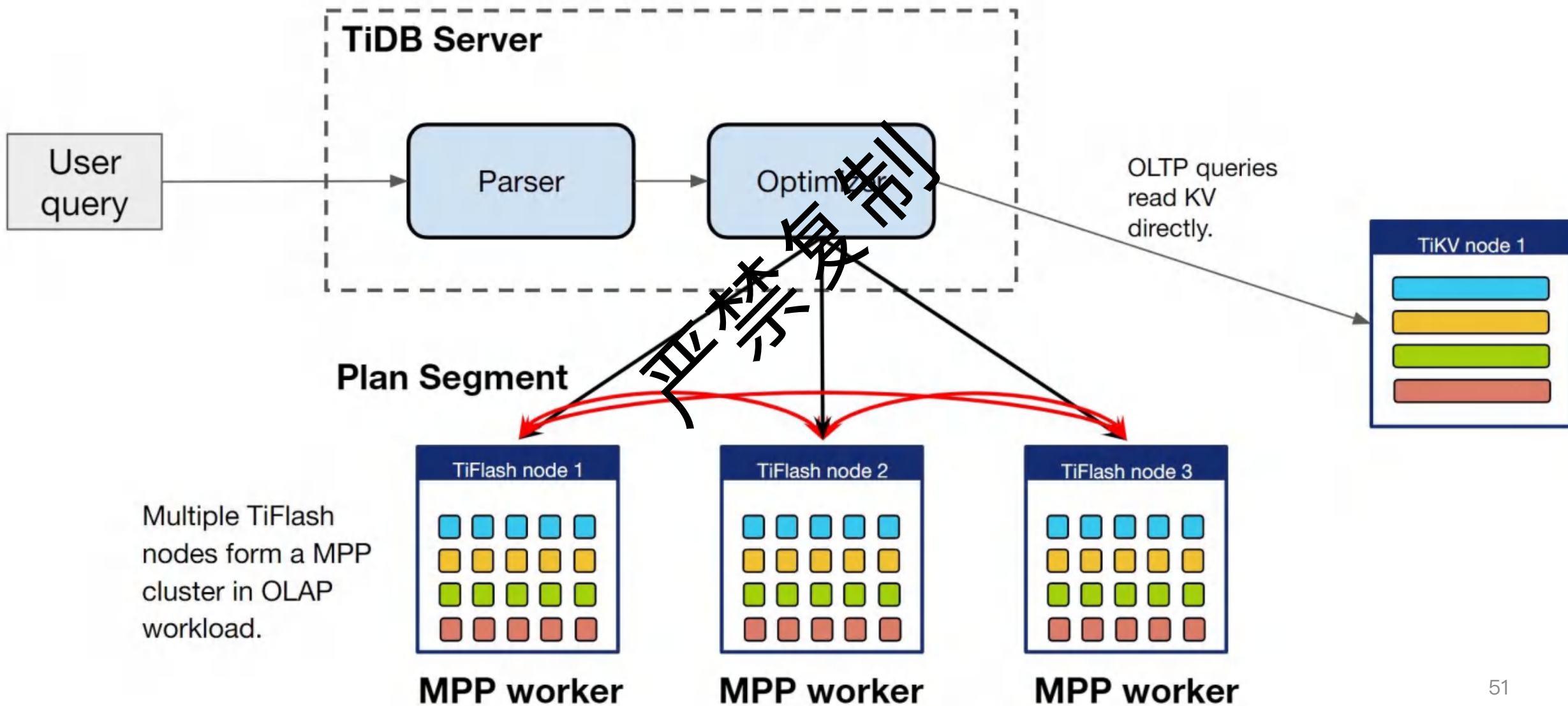
TiDB核心组件

- TiDB Server (SQL 层)
- TiKV (Key-Value 存储引擎)
 - 接收读写请求；
 - 向 PD 汇报状态和心跳
 - 接收 PD 的调度指令。
- PD (调度和管理组件)
 - TiDB Server 和 TiKV 的状态
 - 接收调度指令
- TiFlash (列式存储引擎)
 - 从 TiKV 获取数据同步；
 - 接收 TiDB Server 的分析查询请求。



HTAP混合OLTP OLAP

In MPP mode, TiDB-Server becomes the coordinator.

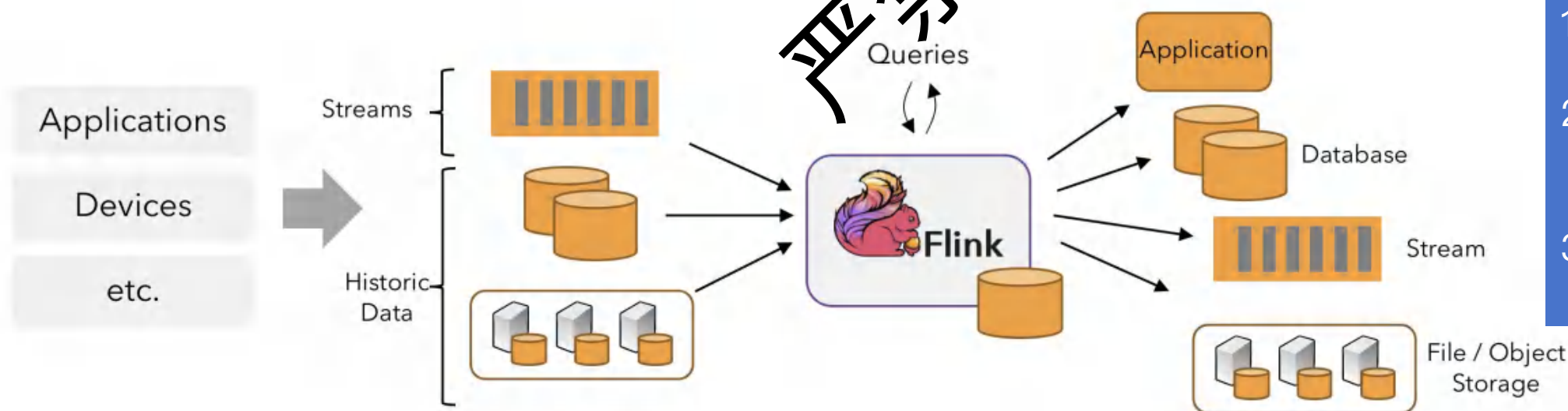




8 复制
格式处理

大数据时代：从数据产生到数据洞察

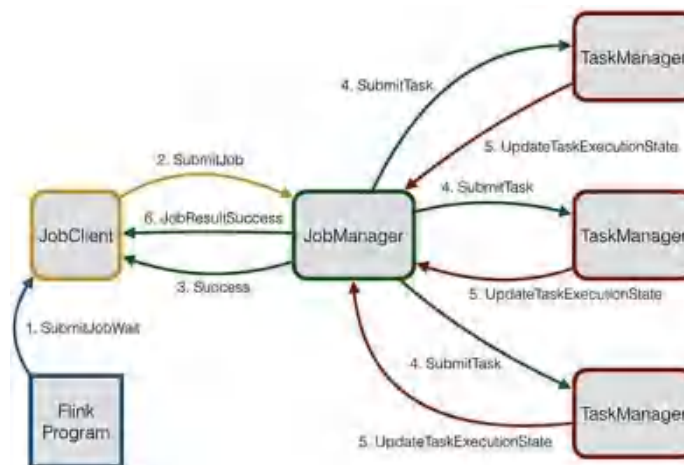
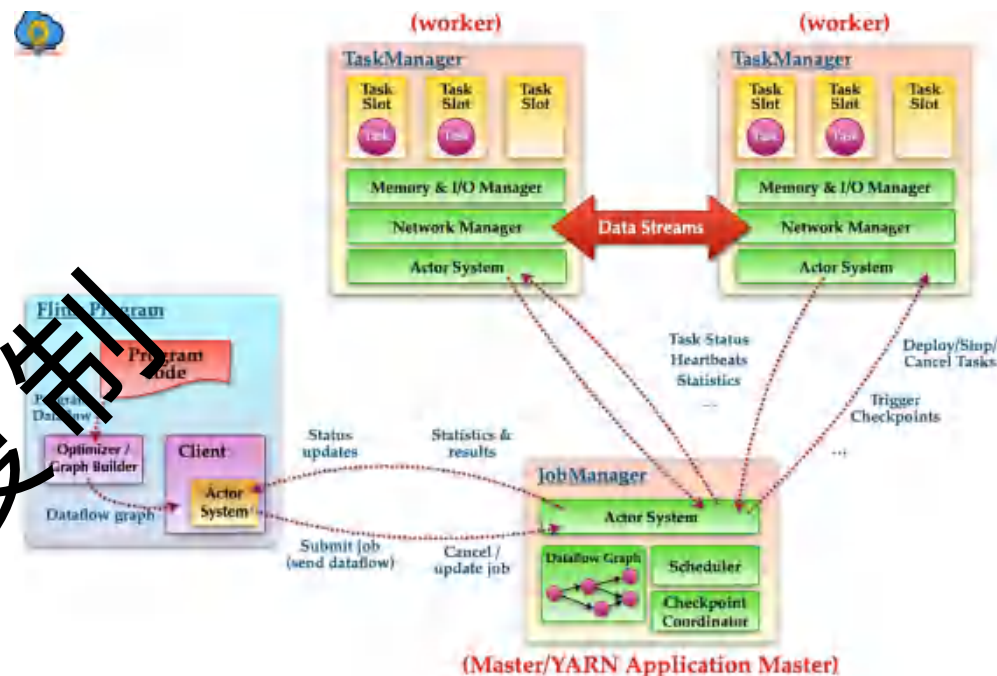
- 数据洪流：线上系统的持续产出
 - Flink, Kafka Streams, Storm, Spark DStreams
- Flink的角色定位
 - 原始数据通常是“脏”的、非结构化的；需要清洗、转换、聚合、富化。
 - 目标：转化为有价值的信息，存储于数据仓库(Data Warehouse, DW)。



1. 实时用户画像更新
2. 实时欺诈检测并将结果写入DW
3. 实时报表生成与OLAP预聚合

Flink核心架构与理念：整体架构

- 核心理念：一切皆是流 (Stream is everything)
 - 批处理是流处理的特例 (Bounded Stream)
 - 统一了流处理和批处理。
- 关键组件概览
 - JobManager (Master): 协调者。负责作业调度、检查点协调、故障恢复。
 - TaskManager (Worker): 执行者。负责执行具体任务(Task)，管理内存和数据流。
 - Client: 提交作业到JobManager。



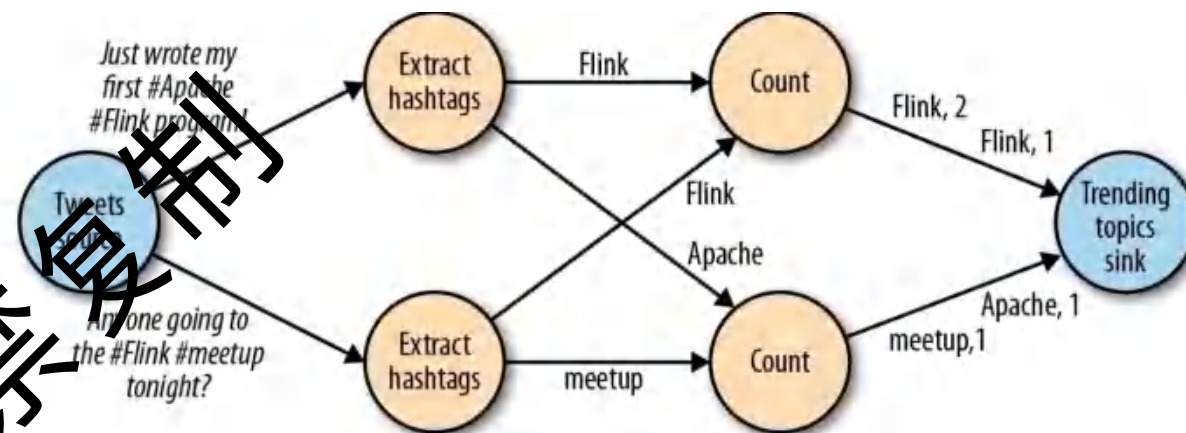
Flink核心架构与理念： 分布式处理过程

- 数据流图 (Dataflow Graph)

- Source (数据源) -> Transformations (转换操作) -> Sink (数据汇)

- 强大的API支持

- DataStream API (核心, 用于流处理)
- DataSet API (用于批处理)
- Table API & SQL (声明式, 简化开发, 统一流批)



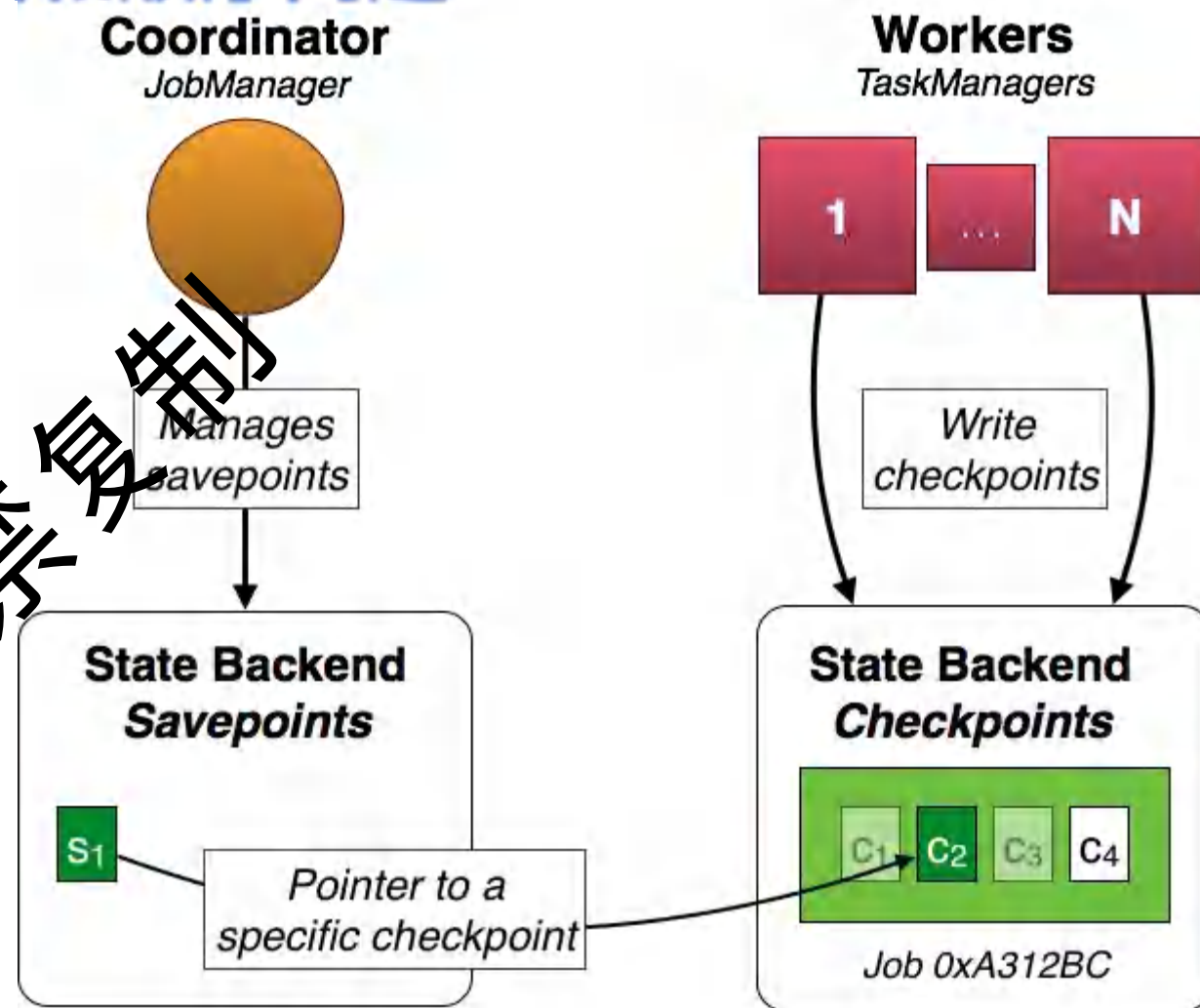
```
// 3. 进行转换操作 (Transformation) - 将每个字符串转换为大写
DataStream<String> upperCaseStream = textStream.map(new MapFunction<String, String>() {
    @Override
    public String map(String value) throws Exception {
        return value.toUpperCase();
    }
});
```

数据正确性的挑战：分布式流处理的“拦路虎”

- 什么是数据正确性 (Processing Semantics)?
 - At-most-once (最多一次)。
 - At-least-once (至少一次)。
 - Exactly-once (精确一次)。这是最理想的状态。
- 分布式环境下的挑战
 - 节点故障 (Node Failures): TaskManager或JobManager宕机。
 - 网络问题 (Network Issues): 消息丢失、延迟、乱序。
 - 背压 (Backpressure): 下游处理慢于上游, 导致数据积压。
 - 无序事件 (Out-of-Order Events): 事件到达顺序与产生顺序不一致 (事件时间 vs. 处理时间)。
- 对于数据仓库而言, 数据不正确意味着什么?
 - At-most-once: 报表数据偏少, 决策失误。
 - At-least-once: 指标重复计算 (如GMV翻倍), 导致严重误判。
 - Exactly-once是目标: 保证进入数据仓库的数据是准确、一致的。

Flink的分布式快照：解决容错和顺序问题

- 核心机制：Checkpointing (分布式快照)
 - Flink的核心容错机制，是实现Exactly-once语义的基石。
 - 原理：JobManager定期向所有Source注入特殊标记 (Checkpoint Barriers)。
 - Barriers随数据流向下游流动。
 - 当一个Operator接收到所有输入流的Barrier时，它会将当前的状态快照到持久化存储 (如HDFS, S3)。
 - Operator处理完Barrier后，将其广播给下游。
 - 当所有Sink都确认了Barrier，一次Checkpoint完成。



Flink vs. Spark (Streaming): 模型的差异

- Spark Streaming (DStream API 微批处理 Micro-batching)
 - 模型：将流数据切分成一系列小的批次 (RDDs) 进行处理。本质是快速批处理。
 - 延迟：受限于Micro-batch的间隔，通常在数百毫秒到秒级。
 - 状态管理：updateStateByKey, mapWithState, 状态存储在RDD中。
 - 数据正确性：可以实现Exactly-once（通过WAL和事务性输出）。
 - 窗口操作：基于时间的窗口，但受微批模型限制。
 - 事件时间处理：支持，但不如Flink原生和灵活。
- Apache Flink (真·流处理 Stream Processing)
 - 模型：逐条处理数据，事件驱动 (Event-at-a-time)。
 - 延迟：可以达到毫秒级甚至亚毫秒级。
 - 状态管理：原生支持，高效且灵活，与Checkpointing紧密集成。
 - 数据正确性：原生设计支持Exactly-once。
 - 窗口操作：非常灵活，支持基于事件时间/处理时间的各种复杂窗口。
 - 事件时间处理：核心特性，处理乱序数据能力强。
- Spark Structured Streaming
 - Spark生态中更新的流处理引擎，更接近Flink的流模型，但底层仍有微批思想的痕迹（Continuous Processing模式在努力接近真流）。
- 总结：Flink是为纯粹的流处理而设计的，在低延迟、事件时间处理和状态管理方面具有天然优势。Spark Streaming (DStream) 本质是微批，而Structured Streaming正在向Flink的真流模型靠拢但各有侧重。

期末考试

- 9. 业务背景：一位资深架构师正在为公司选择一套全面的架构文档方法，既要满足
- 不同团队的详细需求，又要兼顾系统的并发和部署复杂性。
- 题目：相较于 C4 模型，Kruchten 4+1 视图模型在哪些方面提供了更明确或更深入的
- 关注？
- A. 提供了专门的视图来描述并发、分布和同步机制（进程视图）。
- B. 提供了专门的视图来描述软件的物理部署和硬件拓扑（物理视图）。
- C. 提供了专门的视图来描述开发环境的组织和代码结构（开发视图）。
- D. 能够以更低的抽象层次直接展示单个代码文件和类之间的关系。
- 答案：A, B, C
- 解释：* A：Kruchten的进程视图明确关注并发和分布。* B：Kruchten的物理视图明确关注物理部署。* C：Kruchten的开发视图明确关注开发环境和代码组织。* D：这是C4模型（特别是代码视图）的特点，而非Kruchten 4+1的独有优势。* E：这是C4模型的优势，Kruchten 4+1模型通常被认为更为全面和重量级。

THANK YOU

严禁复制

